Abdulmohsen Almunayes

RED ID: 823343580

04/18/2024

LAB 8 REPORT

What does your program do?

My program illuminates an LED by adjusting its brightness using a duty cycle. Initially, the EEPROM reads and stores the current duty cycle. Every time the USB is plugged back into the microcontroller, the LED brightness increases progressively, as the program consistently sets a higher duty cycle than the previously saved value in the EEPROM.

What variables/registers does your program use?

The program employs global variables called address to hold the address using EEMEM in the EEPROM address space and dutyCycle to hold the current duty cycle value. Register B is configured for input-output operations, with pin B5 designated for controlling the LED. Timer-related registers TCCR, OCR, and TIMSK are utilized to configure a timer and its associated interrupts.

What is the main algorithm/logic of your program?

The program begins by configuring the LED's data direction as output. It proceeds to retrieve the value stored in EEPROM address 0 utilizing the eeprom_read_byte function, assigning it to the variable dutyCycle. OCR0A is then determined to ensure a timer period of 5 milliseconds with a prescaler of 1,024. Subsequently, OCR0B is established based on dutyCycle's percentage in relation to OCR0A. Following this, dutyCycle is incremented by 10 if dutyCycle is not 100 and then constrained within the range of 0 to 100. The updated dutyCycle is stored back into EEPROM address 0 utilizing the eeprom_update_byte function. Timer 0 interrupts are employed to toggle the LED on and off correspondingly.

What external hardware connections did you make?

Connections were initially established between the computer and the microcontroller to upload the program. Subsequently, only a power source connection was required for the microcontroller to operate independently.

C CODE

```
#define F_CPU 16000000UL

#include <avr/io.h>
#include <avr/interrupt.h>
#include <avr/eeprom.h>

uint8_t EEMEM address = 0;
uint8_t dutyCycle;

int main(void) {

   DDRB |= (1 << 5);
   Timer0_Init();
   sei();

   dutyCycle = eeprom_read_byte(&address);

   if (dutyCycle >= 100) {
      dutyCycle = 0;
   } else {
      dutyCycle += 10;
   }

   eeprom_update_byte(&address, dutyCycle);
   OCR0B = OCR0A * (dutyCycle / 100.0);

   while (1) {}
}

void Timer0_Init() {
   TCCR0A |= (1 << WGM01);
   TCCR0B |= (1 << CS02) | (1 << CS00);
   OCR0A = 78;
   TIMSK0 |= (1 << OCIE0A) | (1 << OCIE0B);
}

ISR(TIMER0_COMPA_vect) {
   PORTB |= (1 << 5);
}

ISR(TIMER0_COMPB_vect) {
   PORTB &= ~(1 << 5);
}
```